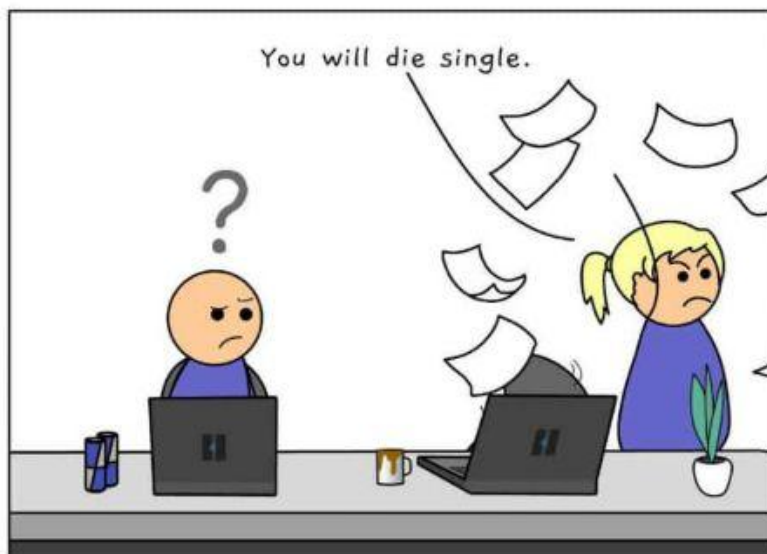
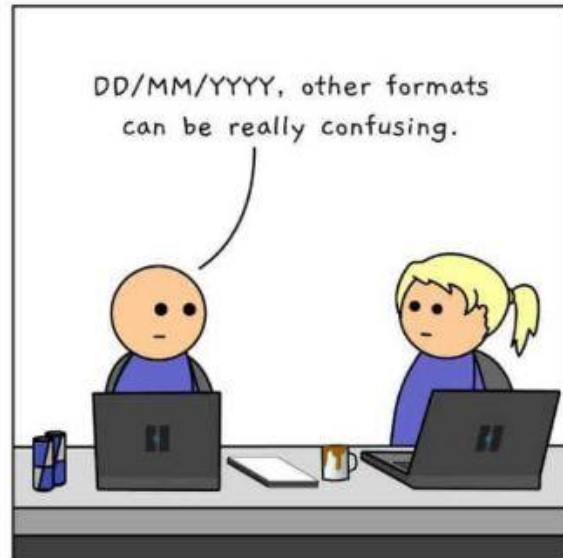
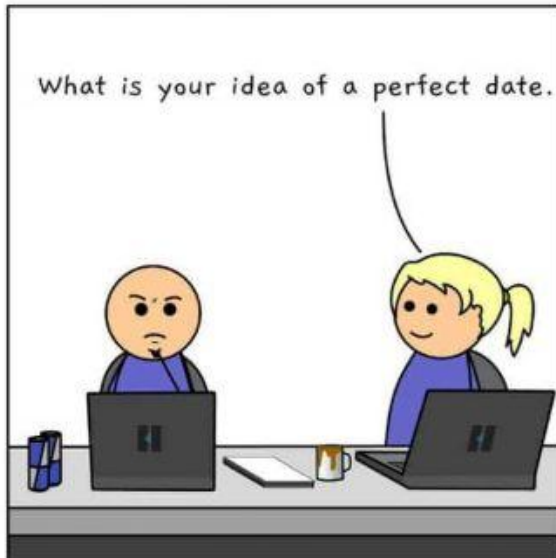


Data Preparation Session 7 Handout

Date and time



The lubridate package in tidyverse is designed for date and time. It has a lot of very intuitive solutions but needs some basic knowledge about date and time objects generally.

The major date and times classes (types) in R:

- Date: stores dates, not time, as number of days since the epoch (1970-01-01). Negative numbers before the epoch.
- POSIXct, POSIXlt: the default times format yyyy-mm-dd HH:MM:SS TZ, POSIXct stores the seconds since the epoch, POSIXlt stores in a list and you can access to them with the \$ symbol
- chron: chronological objects with a series of date/time objects
- difftime: time difference in seconds

The beginning of time is the epoch. It can be very different for different programming languages. In R, it is 1st January 1970, but for Microsoft 0th January 1900.

`origin`

```
[1] "1970-01-01 UTC"
```

`today()`

```
[1] "2026-01-12"
```

`now()`

```
[1] "2026-01-12 10:32:14 CET"
```

We can check the number of days or seconds since the epoch:

`as.numeric(today())`

`as.numeric(now())`

Date construction functions

They create date or time objects.

Example	Function
January 12, 2026	<code>mdy()</code>
2026-01-12	<code>ymd()</code>
12/01/2026	<code>dmy()</code>
January 12, 2026 10:34:21	<code>mdy_hms()</code>
2026-01-12 10:34:21	<code>ymd_hms()</code>
12/01/2026 10:34:21	<code>dmy_hms()</code>

And you can add the time zone (if not the default) using the `tz` argument.

A few examples:

```
mdy("01/12/2026")
mdy("01:12:26")
dmy("12/01/26")
ymd("2026-01-12")
```

And we can include time

```
ymd_hms("2026-01-12 10:30:26")
```

Let's set that to Eastern time

```
ymd_hms("2026-01-12 10:30:26", tz='EST')
```

Extraction (without regex)

To extract different elements, you can use `year()`, `month()`, `day()`, `hour()`, `minute()`, `second()`, but also `wday()`, `yday()` or `leap_year()`.

Let's play with Alan Turing.

```
alan <- mdy_hm("June 23, 1912, 02:15 AM", tz=Europe/London)
```

Which day of the week?

```
wday(alan)
wday(alan, label=TRUE)
wday(alan, label = TRUE, abbr = FALSE) # lubridate is a bit more complex
```

Which semester? AM or PM? Was it a daylight-saving time? Was it a leap_year?

```
semester(alan)
am(alan)
dst(alan)
leap_year(alan)
```

In which year was he 3 years old? Extract year and add.

```
year(alan) + 3
```

Time zones

The time zone is important to know the exact time. How do we handle this problem? using `tz` argument in the time functions, and we can check

What was the time in Tokyo?

```
grep("Tokyo", OlsonNames(), value = TRUE)
```

```
with_tz(alan, "Asia/Tokyo") # Japan standard time
```

```
tz(alan)
```

Rounding

You can round (round_date), round down (floor_date) or round up (ceiling_date) date/time objects to any unit.

```
floor_date(now(), unit = "month") # round down
```

```
floor_date(now(), unit = "year")
```

```
round_date(now(), unit = "hour") # round to nearest unit
```

```
ceiling_date(now(), unit = "minutes") # round up
```

Time difference

There are three classes:

duration: time difference, WITHOUT looking at the calendar: dyears, dweeks, ddays (NO dmonths)

period: time difference, WITH looking at the calendar: years, months, weeks, days

interval: has a starting and an end point (so not the difference, but the two limits)

Time differences

```
ddays(370)
```

```
days(370)
```

Up to days, everything is easy, like one minute = 60 seconds, and one hour = 60 minutes.

How long is a month? How long is a year?

```
LunarOrbit <- ddays(29) + dhours(12) + dminutes(44)
```

We can do additions:

```
ymd("2024-01-31") + LunarOrbit
```

```
ymd("2026-01-31") + LunarOrbit
```

```
dmy("28 February, 2024") + years(1)
```

```
dmy("29 February, 2024") + years(1)
```

```
dmy("29 February, 2024") + dyears(1) # One "dyear" is 365.25 days
```

```
dmy("29 February, 2024") + months(1)
```

```
dmy("31 January, 2024") + months(1)
```

```
dmy("31 January, 2024") + dmonths(1) # One "dmonth" is 1/12 "dyear"
```

Finally, how old is Alan now?

```
alandiff <- interval(alan, now())
```

```
alandiff/years(1) # periods understand leap years
```

```
alandiff/dyears(1) # durations don't
```

```
round(alandiff/dyears(1))
```